



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Дальневосточный федеральный университет»
(ДВФУ)**

**ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ
ТЕХНОЛОГИЙ**

Лабораторная работа 1

Дисциплина «Теория вероятностей и математическая статистика»

Студент группы Б9123-01.03.02ии

Есакова Елизавета Михайловна

г. Владивосток

2025

Лабораторная в гугл коллабе:

https://colab.research.google.com/drive/1PjBOxePuCMYKoRBss_Ig8i61ZxBxb3Ea#scrollTo=oI4Mw2BwbjNr

Параметры распределения:

```
uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}
```

Равномерное распределение

loc – левая граница интервала (a)

scale – длинна интервала (b-a)

Плотность вероятности:

$$f(x) = \frac{1}{b-a} (a \leq x \leq b)$$

(в коде a=5, b=15)

Распределение Бернулли:

p – вероятность успеха (X=1)

Функция вероятности: $P(X = 1) = p, P(X = 0) = 1 - p$

(в коде p=0.7)

Биномиальное распределение:

n- количество испытаний

p – вероятность успеха в каждом испытании

Функция вероятности: $P(X = k) = C_n^k * p^k * (1 - p)^{n-k}$

(в коде n=20, p=0.4)

Нормальное распределение:

loc (μ) – среднее значение

scale (σ) – стандартное отклонение

Формула плотности:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

(в коде $\mu = 5, \sigma = 2$)

Функции для расчета статистик:

Формула выборочного среднего:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Где n – объем выборки, а x_i – элемент выборки

В коде:

Без numpy:

```
def sample_mean(data):  
    return sum(data) / len(data)
```

С numpy:

```
numpy_mean = np.mean(sample)
```

Дисперсия:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

В коде:

Без numpy:

```
def sample_variance(data):  
    mean = sample_mean(data)  
    return sum((x - mean) ** 2 for x in data) / len(data)
```

С numpy:

```
numpy_variance = np.var(sample, ddof=0)
```

Стандартное отклонение:

$$\sigma = \sqrt{\sigma^2}$$

В коде:

Без numpy:

```
def sample_std(data):  
    return sample_variance(data) ** 0.5
```

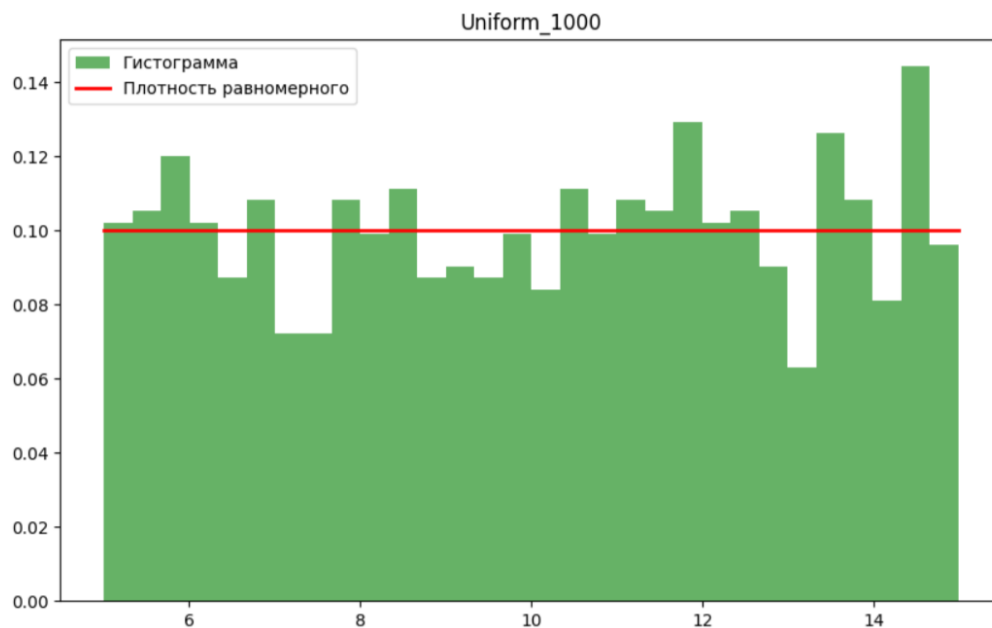
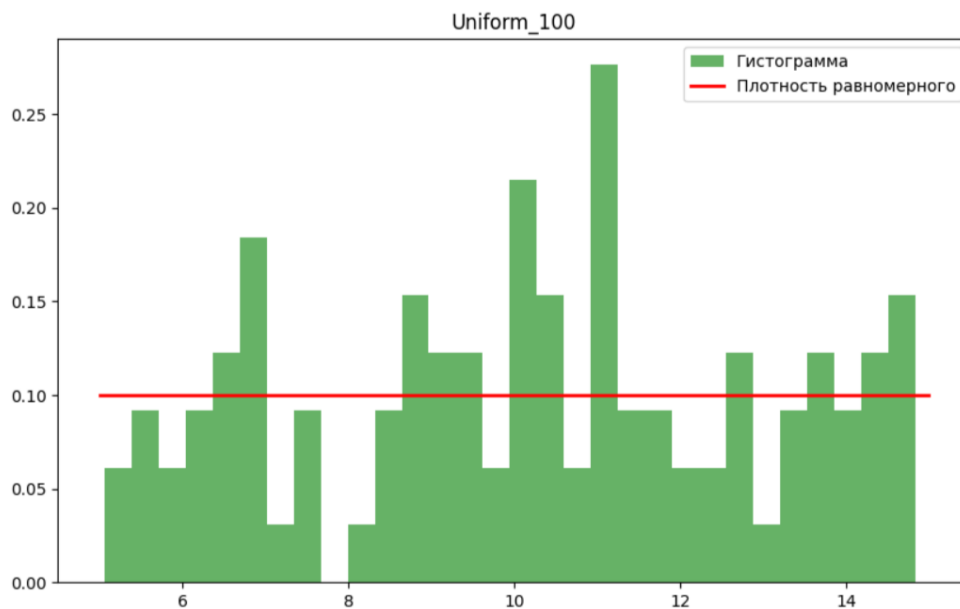
С numpy:

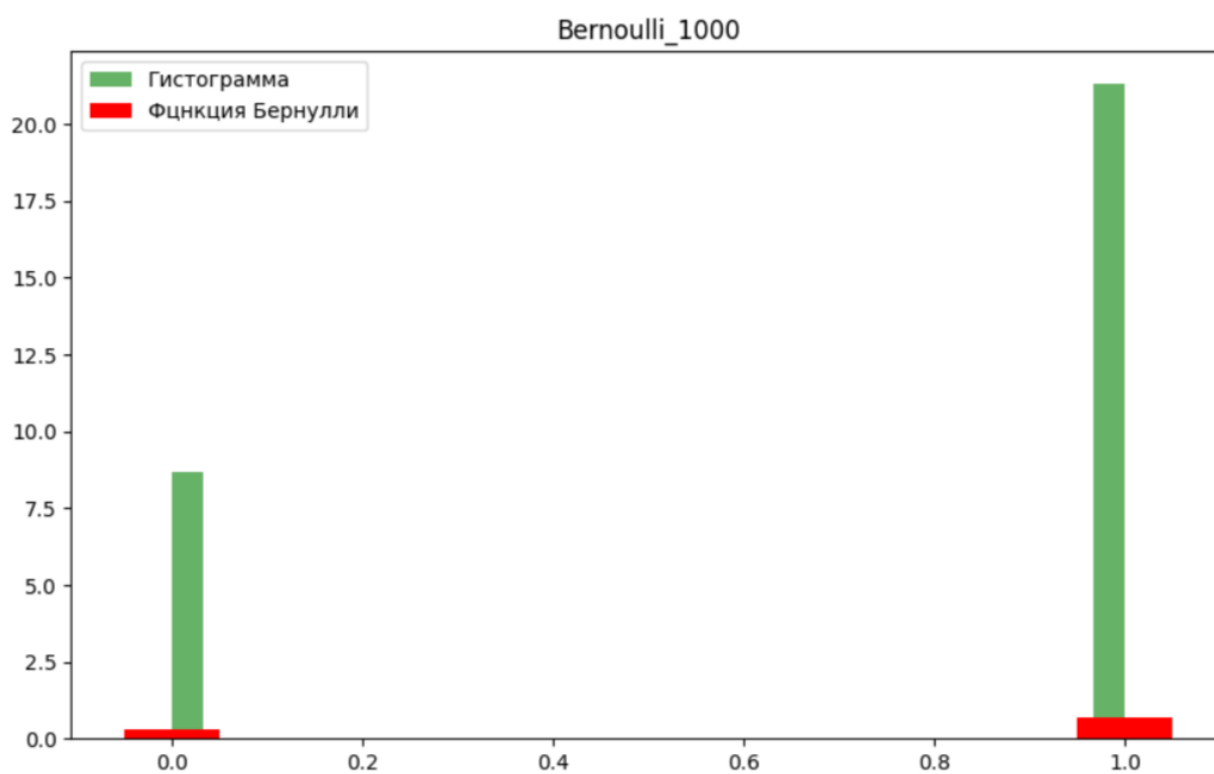
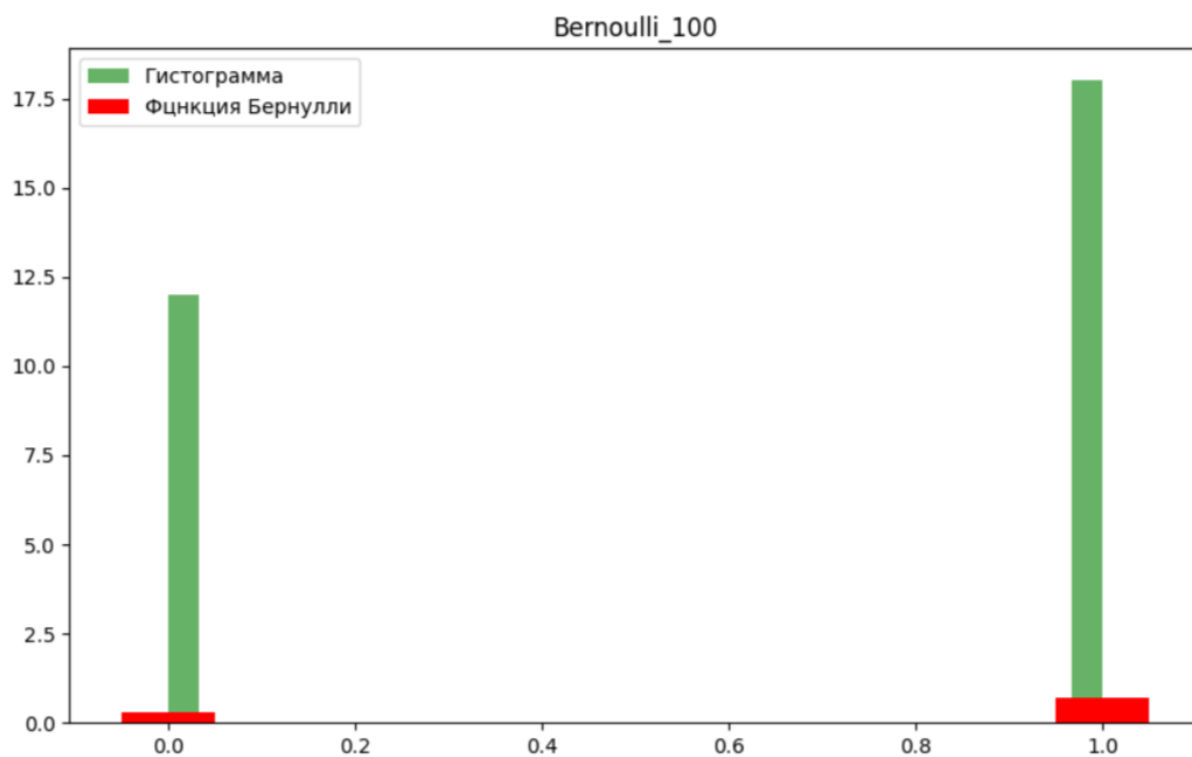
```
numpy_std = np.std(sample, ddof=0)
```

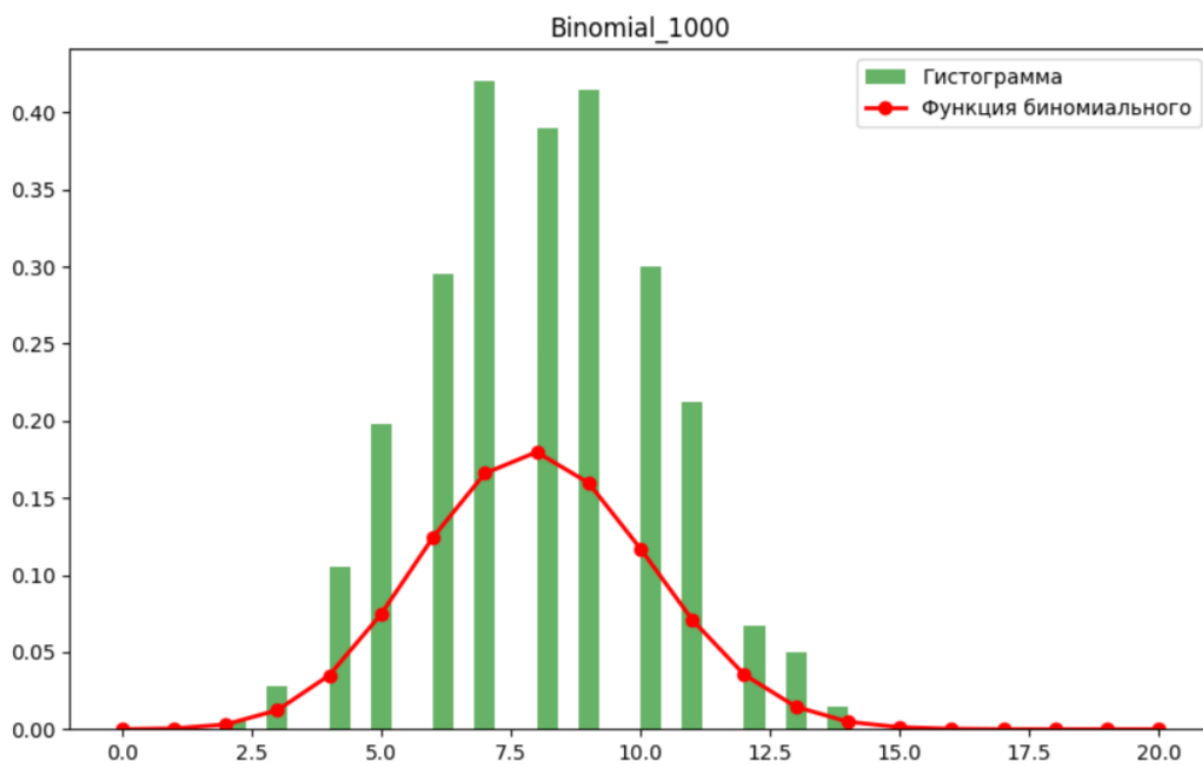
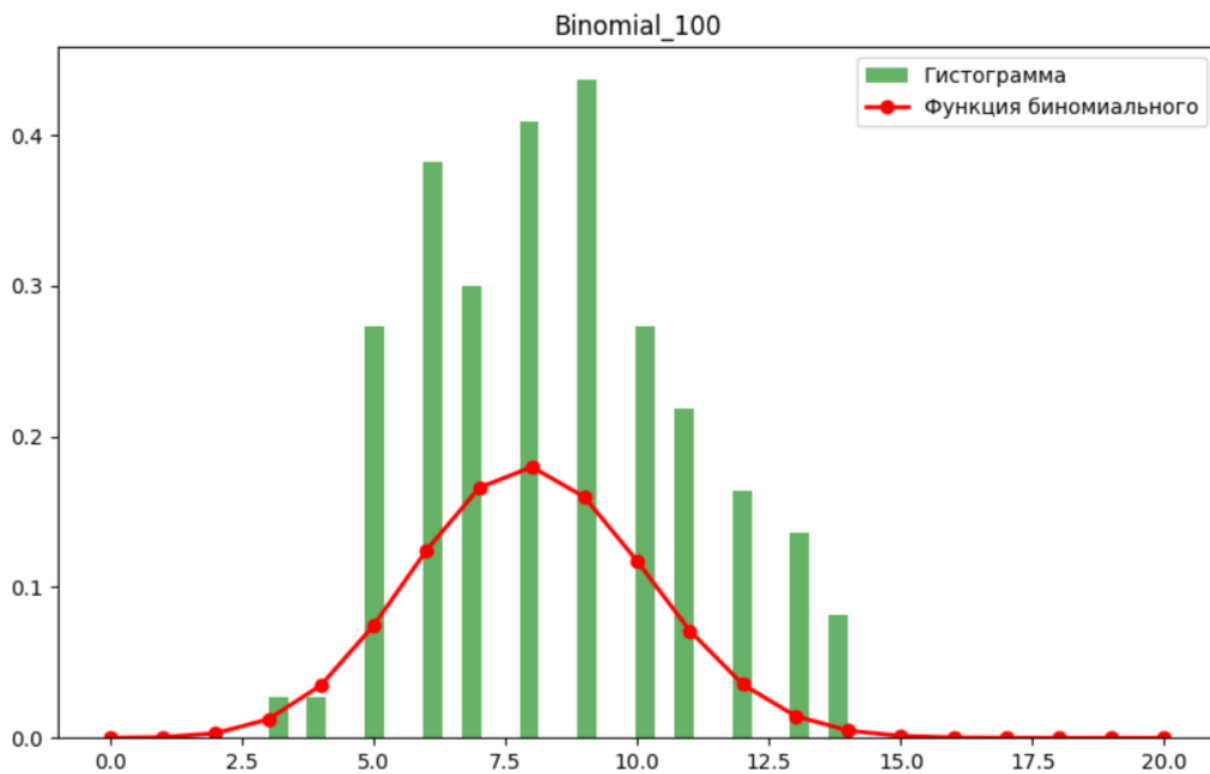
Вывод результатов:

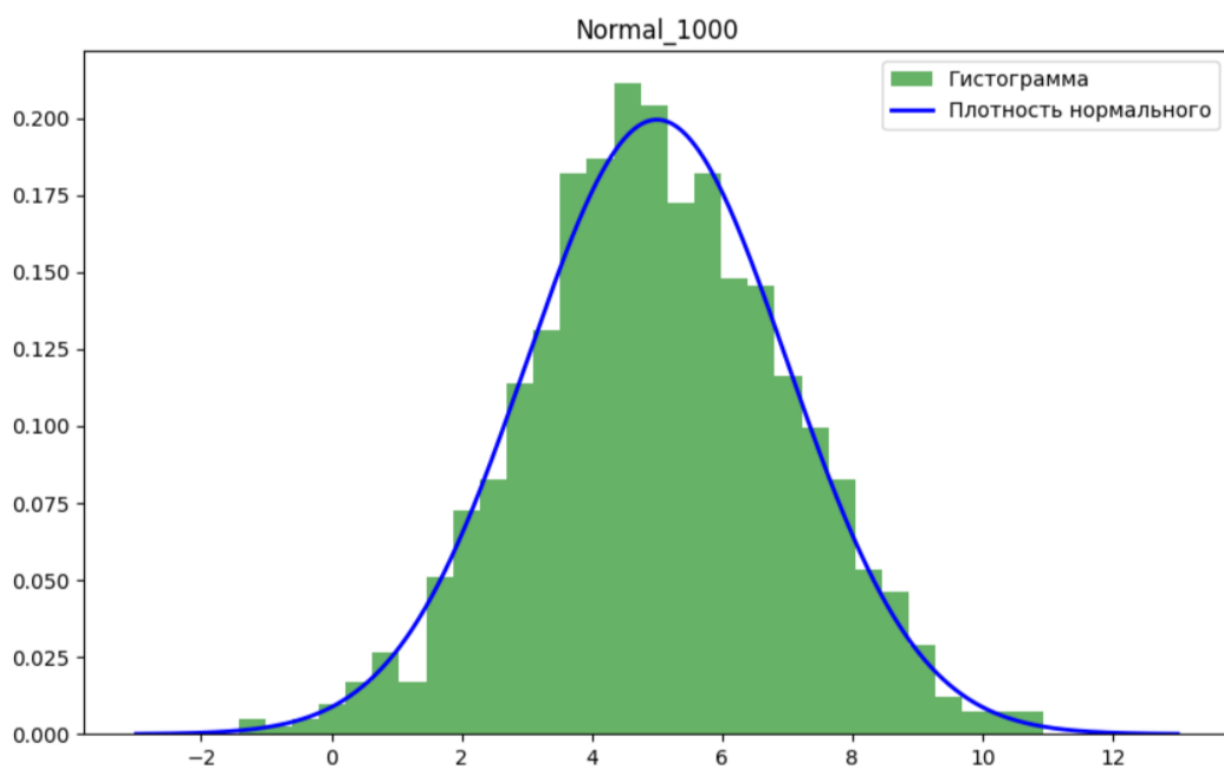
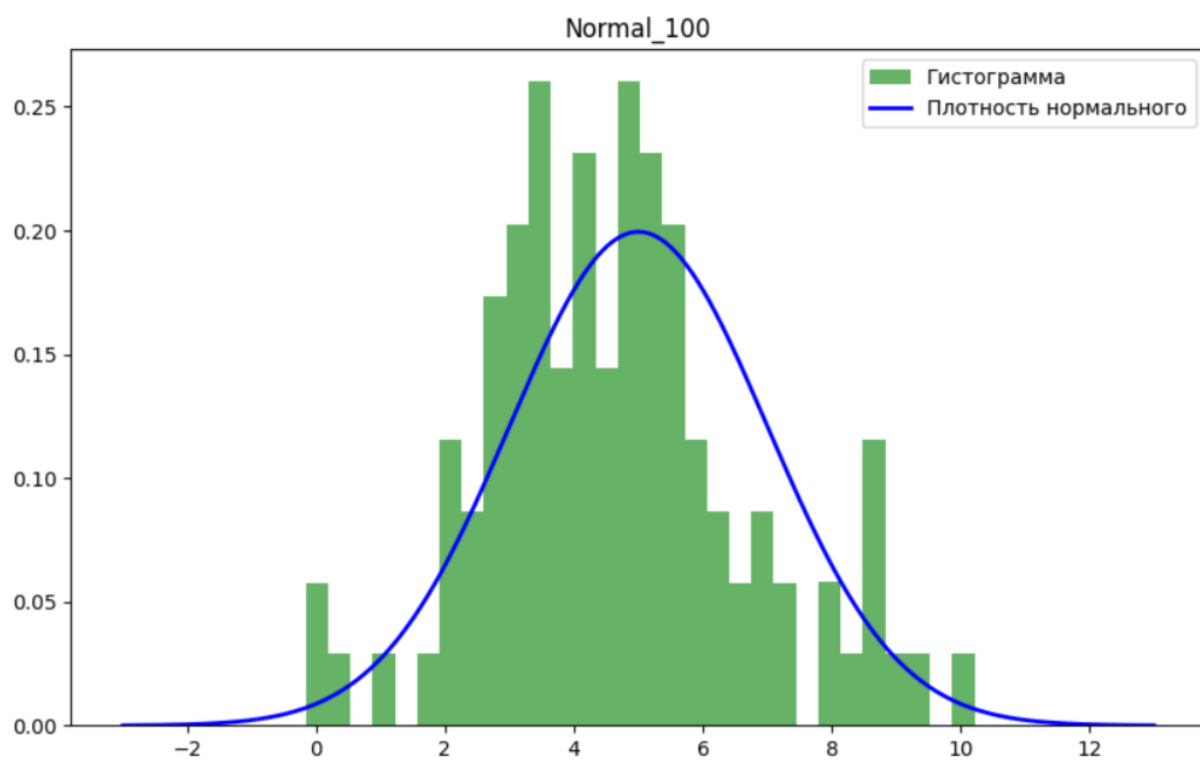
	Выборка	Среднее	Дисперсия	Стандартное отклонение	\
0	Uniform_100	10.198723	7.468416	2.732840	
1	Uniform_1000	10.073776	8.525664	2.919874	
2	Bernoulli_100	0.600000	0.240000	0.489898	
3	Bernoulli_1000	0.710000	0.205900	0.453762	
4	Binomial_100	8.490000	6.309900	2.511951	
5	Binomial_1000	8.029000	4.928159	2.219946	
6	Normal_100	4.645445	4.058136	2.014481	
7	Normal_1000	5.011657	4.022216	2.005546	

	Среднее (numpy)	Дисперсия (numpy)	Стандартное отклонение (numpy)
0	10.198723	7.468416	2.732840
1	10.073776	8.525664	2.919874
2	0.600000	0.240000	0.489898
3	0.710000	0.205900	0.453762
4	8.490000	6.309900	2.511951
5	8.029000	4.928159	2.219946
6	4.645445	4.058136	2.014481
7	5.011657	4.022216	2.005546









Полный код:

```
import scipy.stats as stats
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(4)

uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}

samples = {
    'Uniform_100': stats.uniform.rvs(size=100, **uniform_params),
    'Uniform_1000': stats.uniform.rvs(size=1000, **uniform_params),
    'Bernoulli_100': stats.bernoulli.rvs(size=100, **bernoulli_params),
    'Bernoulli_1000': stats.bernoulli.rvs(size=1000, **bernoulli_params),
    'Binomial_100': stats.binom.rvs(size=100, **binomial_params),
    'Binomial_1000': stats.binom.rvs(size=1000, **binomial_params),
    'Normal_100': stats.norm.rvs(size=100, **normal_params),
    'Normal_1000': stats.norm.rvs(size=1000, **normal_params)
}

def sample_mean(data):
    return sum(data) / len(data)

def sample_variance(data):
    mean = sample_mean(data)
    return sum((x - mean) ** 2 for x in data) / len(data)

def sample_std(data):
    return sample_variance(data) ** 0.5

results = []

for name, sample in samples.items():

    nonum_mean = sample_mean(sample)
    nonum_variance = sample_variance(sample)
    nonum_std = sample_std(sample)

    numpy_mean = np.mean(sample)
    numpy_variance = np.var(sample, ddof=0)
    numpy_std = np.std(sample, ddof=0)

    results.append({
```



```

        'Выборка': name,
        'Среднее': nonum_mean,
        'Дисперсия': nonum_variance,
        'Стандартное отклонение': nonum_std,
        'Среднее (numpy)': numpy_mean,
        'Дисперсия (numpy)': numpy_variance,
        'Стандартное отклонение (numpy)': numpy_std
    })

df_results = pd.DataFrame(results)
print(df_results)

for name, sample in samples.items():
    plt.figure(figsize=(10, 6))
    plt.hist(sample, bins=30, density=True, alpha=0.6, color='g',
label='Гистограмма')

    if 'Uniform' in name:
        loc, scale = uniform_params['loc'], uniform_params['scale']
        x = np.linspace(loc, loc + scale, 1000)
        y = stats.uniform.pdf(x, loc=loc, scale=scale)
        plt.plot(x, y, 'r-', lw=2, label='Плотность равномерного')
    elif 'Bernoulli' in name:
        p = bernoulli_params['p']
        x = [0, 1]
        y = [1-p, p]
        plt.bar(x, y, width=0.1, color='r', label='Функция Бернулли')
    elif 'Binomial' in name:
        n, p = binomial_params['n'], binomial_params['p']
        x = np.arange(0, n+1)
        y = stats.binom.pmf(x, n, p)
        plt.plot(x, y, 'ro-', lw=2, label='Функция биномиального')
    elif 'Normal' in name:
        loc, scale = normal_params['loc'], normal_params['scale']
        x = np.linspace(loc - 4*scale, loc + 4*scale, 1000)
        y = stats.norm.pdf(x, loc=loc, scale=scale)
        plt.plot(x, y, 'b-', lw=2, label='Плотность нормального')

plt.title(f'{name}')
plt.legend()
plt.show()

```